

ReGo - Corporate Travel Management System

Complete Technical Documentation & Implementation Guide

Version: 1.0

Date: November 12, 2025

Team: 5 Members (4 Developers + 1 QA)

Timeline: 5 Weeks with Weekly Milestones



Table of Contents

1. Executive Summary
 2. System Architecture
 3. Technology Stack
 4. Database Design
 5. API Specifications
 6. Frontend Architecture
 7. Authentication & Security
 8. AI & OCR Integration
 9. Complete Workflows
 10. Development Roadmap
 11. Team Assignment & Responsibilities
 12. Testing Strategy
 13. Deployment Guide
 14. Additional Features & Edge Cases
-

1. Executive Summary

1.1 Project Overview

ReGo (Reise + Ergo) eliminates the chaos of corporate travel management by providing a unified platform where employees submit travel requests, documents get automatically verified using AI, approvals happen in real-time, and expenses are processed seamlessly.

1.2 Core Problems Solved

- **✗ Manual Processes** → **✓** Automated workflows
- **✗ Lost Documents** → **✓** Secure cloud storage with OCR

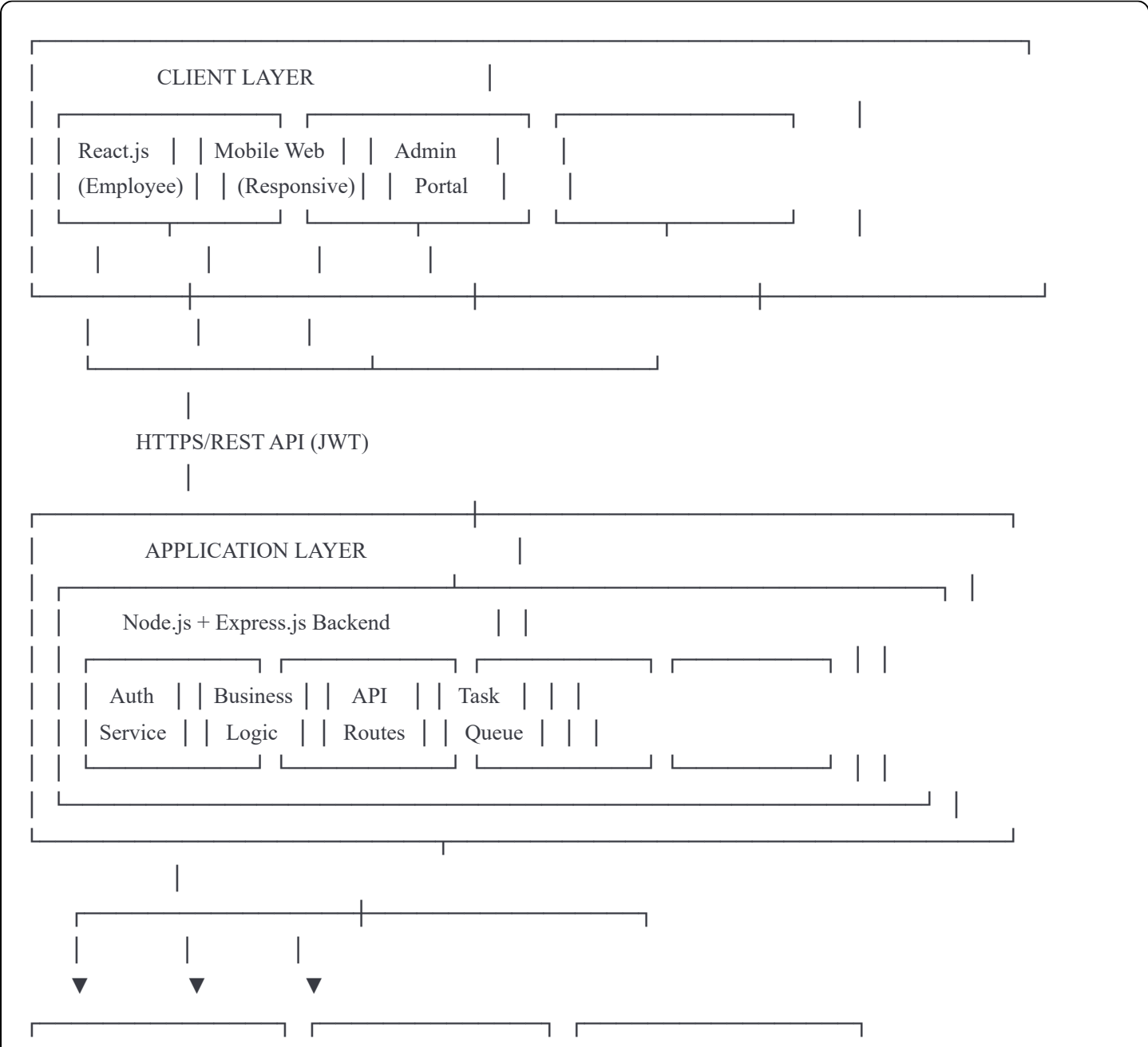
- **✗ Delayed Approvals** → **✓ Real-time multi-level approval**
- **✗ Slow Reimbursements** → **✓ Fast digital expense processing**
- **✗ No Transparency** → **✓ Complete visibility for all stakeholders**

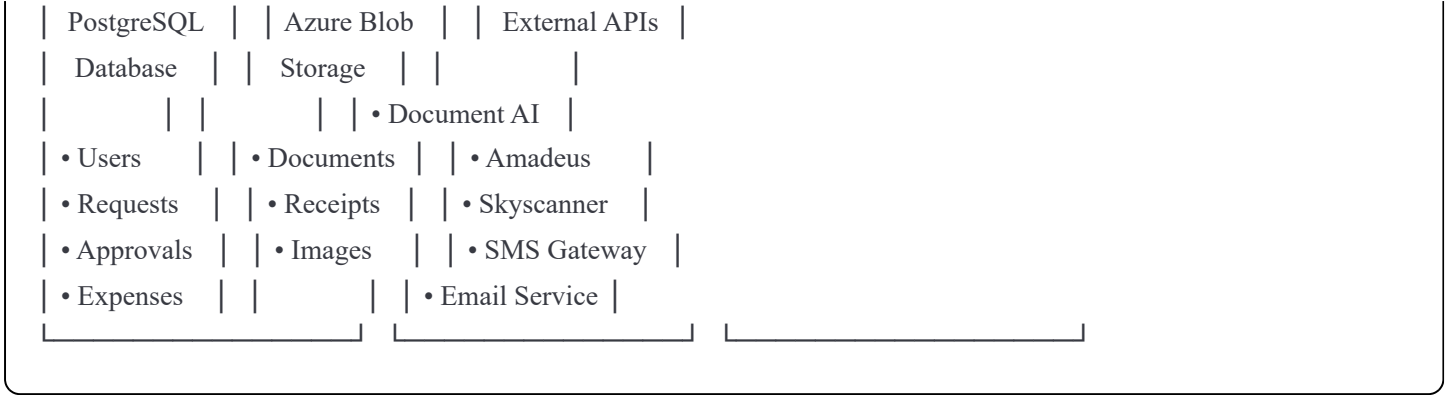
1.3 Key Differentiators

- **AI-Powered OCR:** Azure Document Intelligence auto-verifies passports, visas, and receipts
- **Smart Recommendations:** AI suggests optimal flight and hotel options across price ranges
- **Anomaly Detection:** ML flags suspicious expenses automatically
- **Complete Audit Trail:** Every action logged for compliance
- **Mobile-Friendly:** Responsive design for on-the-go access

2. System Architecture

2.1 High-Level Architecture Diagram





2.2 Component Breakdown

Frontend Components:

- Employee Portal (request submission, expense filing)
- Approver Dashboard (review & approve)
- Travel Desk Portal (booking management)
- Finance Dashboard (reimbursement processing)
- Admin Panel (user & system management)

Backend Services:

- Authentication Service (SSO, JWT)
- Travel Request Service (CRUD operations)
- Approval Workflow Engine (orchestrates multi-level approvals)
- Document Service (OCR, validation, storage)
- Booking Service (flight/hotel API integration)
- Expense Service (submission & reimbursement)
- Notification Service (email, SMS, in-app)
- AI Recommendation Engine (flight/hotel suggestions)

3. Technology Stack

3.1 Complete Stack Overview

Layer	Technology	Version	Purpose
Frontend	React.js	18.3.x	UI framework
	Material-UI (MUI)	5.x	Component library
	Redux Toolkit	2.x	State management
	React Router	6.x	Routing
	React Query	5.x	Server state

Layer	Technology	Version	Purpose
	Formik + Yup	Latest	Form handling
	Axios	1.x	HTTP client
	Chart.js	4.x	Charts/graphs
Backend	Node.js	20.x LTS	Runtime
	Express.js	4.x	Web framework
	Prisma ORM	5.x	Database ORM
	JWT	9.x	Authentication
	bcrypt	5.x	Password hashing
	multer	1.x	File uploads
	winston	3.x	Logging
	joi	17.x	Validation
Database	PostgreSQL	15.x	Primary database
Cloud	Azure App Service	-	Hosting
	Azure Blob Storage	-	File storage
	Azure PostgreSQL	-	Managed DB
	Azure Document Intelligence	-	OCR
	Azure Communication Services	-	SMS/Email
APIs	Amadeus/Skyscanner	-	Flight data
	Booking.com API	-	Hotel data
	Microsoft Graph API	-	Outlook emails

3.2 Development Tools

- **IDE:** VS Code
- **API Testing:** Postman
- **Version Control:** GitHub
- **CI/CD:** GitHub Actions
- **Database Tool:** pgAdmin
- **Containerization:** Docker (optional)

4. Database Design

4.1 Entity Relationship Overview



users (1) ————< (M) expenses

travel_requests (1) ————< (M) approvals

travel_requests (1) ————< (M) documents

travel_requests (1) ————< (M) bookings

travel_requests (1) ————< (M) expenses

travel_requests (1) ————< (M) ai_recommendations

documents (1) ———— (1) expenses (receipt)

4.2 Core Tables Schema

Users Table

sql

```
CREATE TABLE users (  
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  phone VARCHAR(20),  
  role VARCHAR(50) NOT NULL CHECK (role IN (  
    'EMPLOYEE', 'MANAGER', 'AVP', 'SVP', 'CHRO',  
    'TRAVEL_DESK', 'FINANCE', 'ADMIN'  
  )),  
  department VARCHAR(100),  
  employee_id VARCHAR(50) UNIQUE,  
  reporting_manager_id UUID REFERENCES users(user_id),  
  is_active BOOLEAN DEFAULT true,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  last_login TIMESTAMP,  
  profile_image_url TEXT  
);  
  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_role ON users(role);
```

Travel Requests Table

sql

```

CREATE TABLE travel_requests (
  request_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(user_id),
  request_number VARCHAR(50) UNIQUE NOT NULL, -- TR-YYYY-XXXXX

  -- Travel Details
  travel_type VARCHAR(50) NOT NULL CHECK (travel_type IN ('DOMESTIC', 'INTERNATIONAL')),
  purpose TEXT NOT NULL,
  destination_country VARCHAR(100) NOT NULL,
  destination_city VARCHAR(100) NOT NULL,
  departure_date DATE NOT NULL,
  return_date DATE NOT NULL,
  duration_days INTEGER GENERATED ALWAYS AS (return_date - departure_date) STORED,

  -- Cost
  estimated_cost DECIMAL(10, 2),
  approved_budget DECIMAL(10, 2),
  actual_cost DECIMAL(10, 2),
  currency VARCHAR(3) DEFAULT 'INR',

  -- Status
  status VARCHAR(50) NOT NULL DEFAULT 'DRAFT' CHECK (status IN (
    'DRAFT', 'SUBMITTED', 'MANAGER_REVIEW', 'AVP_REVIEW',
    'SVP_REVIEW', 'CHRO_REVIEW', 'APPROVED', 'REJECTED',
    'BOOKING_IN_PROGRESS', 'BOOKED', 'COMPLETED', 'CANCELLED'
  )),

  current_approver_id UUID REFERENCES users(user_id),

  -- Additional
  special_requirements TEXT,
  emergency_contact_name VARCHAR(100),
  emergency_contact_phone VARCHAR(20),
  is_urgent BOOLEAN DEFAULT false,

  -- Timestamps
  submitted_at TIMESTAMP,
  approved_at TIMESTAMP,
  completed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_travel_requests_user ON travel_requests(user_id);

```

```
CREATE INDEX idx_travel_requests_status ON travel_requests(status);
CREATE INDEX idx_travel_requests_dates ON travel_requests(departure_date, return_date);
```

Approvals Table

```
sql

CREATE TABLE approvals (
  approval_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL REFERENCES travel_requests(request_id) ON DELETE CASCADE,
  approver_id UUID NOT NULL REFERENCES users(user_id),
  approval_level INTEGER NOT NULL CHECK (approval_level BETWEEN 1 AND 4),
  -- 1: Manager, 2: AVP, 3: SVP, 4: CHRO

  status VARCHAR(50) NOT NULL DEFAULT 'PENDING' CHECK (status IN (
    'PENDING', 'APPROVED', 'REJECTED', 'DELEGATED'
  )),

  comments TEXT,
  approved_budget DECIMAL(10, 2),
  conditions TEXT,

  reviewed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE(request_id, approval_level)
);

CREATE INDEX idx_approvals_request ON approvals(request_id);
CREATE INDEX idx_approvals_approver ON approvals(approver_id, status);
```

Documents Table

```
sql
```

```

CREATE TABLE documents (
  document_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL REFERENCES travel_requests(request_id) ON DELETE CASCADE,
  uploaded_by UUID NOT NULL REFERENCES users(user_id),

  document_type VARCHAR(50) NOT NULL CHECK (document_type IN (
    'PASSPORT', 'VISA', 'FLIGHT_TICKET', 'HOTEL_BOOKING',
    'INSURANCE', 'RECEIPT', 'INVOICE', 'OTHER'
  )),
  file_name VARCHAR(255) NOT NULL,
  file_size_bytes BIGINT,
  mime_type VARCHAR(100),
  blob_storage_url TEXT NOT NULL,

  -- OCR Processing
  ocr_status VARCHAR(50) DEFAULT 'PENDING' CHECK (ocr_status IN (
    'PENDING', 'PROCESSING', 'SUCCESS', 'FAILED', 'MANUAL_REVIEW'
  )),
  ocr_extracted_data JSONB,
  ocr_confidence_score DECIMAL(5, 2),
  ocr_processed_at TIMESTAMP,

  -- Verification
  verification_status VARCHAR(50) DEFAULT 'PENDING' CHECK (verification_status IN (
    'PENDING', 'VERIFIED', 'REJECTED', 'NEEDS_REUPLOAD'
  )),
  verified_by UUID REFERENCES users(user_id),
  verification_comments TEXT,
  verified_at TIMESTAMP,

  upload_attempt INTEGER DEFAULT 1,

  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_documents_request ON documents(request_id);
CREATE INDEX idx_documents_status ON documents(ocr_status, verification_status);

```

Bookings Table

```
sql
```



```
CREATE TABLE bookings (  
  booking_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  request_id UUID NOT NULL REFERENCES travel_requests(request_id) ON DELETE CASCADE,  
  booked_by UUID NOT NULL REFERENCES users(user_id),  
  
  booking_type VARCHAR(50) NOT NULL CHECK (booking_type IN (  
    'FLIGHT', 'HOTEL', 'VISA', 'INSURANCE', 'CAR_RENTAL', 'TRAIN'  
  )),  
  
  booking_details JSONB NOT NULL,  
  -- Example: {"airline": "Air India", "pnr": "ABC123", ...}  
  
  cost DECIMAL(10, 2) NOT NULL,  
  currency VARCHAR(3) DEFAULT 'INR',  
  
  status VARCHAR(50) DEFAULT 'CONFIRMED' CHECK (status IN (  
    'PENDING', 'CONFIRMED', 'CANCELLED', 'MODIFIED'  
  )),  
  
  confirmation_number VARCHAR(100),  
  vendor_name VARCHAR(200),  
  
  booking_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  travel_date TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Expenses Table

```
sql
```

```

CREATE TABLE expenses (
  expense_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL REFERENCES travel_requests(request_id) ON DELETE CASCADE,
  submitted_by UUID NOT NULL REFERENCES users(user_id),

  expense_category VARCHAR(50) NOT NULL CHECK (expense_category IN (
    'FLIGHT', 'HOTEL', 'MEALS', 'LOCAL_TRANSPORT',
    'TAXI', 'FUEL', 'PARKING', 'VISA_FEES',
    'INSURANCE', 'MISCELLANEOUS'
  )),

  description TEXT NOT NULL,
  amount DECIMAL(10, 2) NOT NULL,
  currency VARCHAR(3) DEFAULT 'INR',
  converted_amount DECIMAL(10, 2),
  exchange_rate DECIMAL(10, 4),

  expense_date DATE NOT NULL,

  receipt_document_id UUID REFERENCES documents(document_id),
  vendor_name VARCHAR(200),

  status VARCHAR(50) DEFAULT 'SUBMITTED' CHECK (status IN (
    'DRAFT', 'SUBMITTED', 'UNDER_REVIEW', 'APPROVED',
    'REJECTED', 'REIMBURSED'
  )),

  approved_by UUID REFERENCES users(user_id),
  approved_amount DECIMAL(10, 2),
  approval_comments TEXT,
  approved_at TIMESTAMP,

  reimbursement_status VARCHAR(50) DEFAULT 'PENDING',
  reimbursed_amount DECIMAL(10, 2),
  reimbursement_date DATE,
  transaction_reference VARCHAR(100),

  submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Notifications Table

```
sql
```

```

CREATE TABLE notifications (
  notification_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(user_id),

  notification_type VARCHAR(50) NOT NULL,
  title VARCHAR(255) NOT NULL,
  message TEXT NOT NULL,

  related_entity_type VARCHAR(50),
  related_entity_id UUID,

  is_read BOOLEAN DEFAULT false,
  read_at TIMESTAMP,

  delivery_method VARCHAR(50),
  delivery_status VARCHAR(50) DEFAULT 'PENDING',

  sent_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

AI Recommendations Table

sql

```

CREATE TABLE ai_recommendations (
  recommendation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID NOT NULL REFERENCES travel_requests(request_id),

  recommendation_type VARCHAR(50) NOT NULL CHECK (recommendation_type IN (
    'FLIGHT', 'HOTEL', 'ITINERARY', 'BUDGET'
  )),

  recommendation_data JSONB NOT NULL,
  confidence_score DECIMAL(5, 2),

  was_selected BOOLEAN DEFAULT false,
  selected_option_index INTEGER,
  user_feedback TEXT,

  generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  expires_at TIMESTAMP
);

```

4.3 Sample Data for Testing

```
sql

-- Sample users
INSERT INTO users (email, password_hash, first_name, last_name, role, department, employee_id) VALUES
('john.doe@company.com', '$2b$10$hashedpassword', 'John', 'Doe', 'EMPLOYEE', 'Engineering', 'EMP001'),
('sarah.manager@company.com', '$2b$10$hashedpassword', 'Sarah', 'Smith', 'MANAGER', 'Engineering', 'MGR001'),
('mike.avp@company.com', '$2b$10$hashedpassword', 'Mike', 'Johnson', 'AVP', 'Operations', 'AVP001'),
('lisa.svp@company.com', '$2b$10$hashedpassword', 'Lisa', 'Brown', 'SVP', 'Operations', 'SVP001'),
('david.chro@company.com', '$2b$10$hashedpassword', 'David', 'Wilson', 'CHRO', 'HR', 'CHRO001'),
('travel.desk@company.com', '$2b$10$hashedpassword', 'Travel', 'Coordinator', 'TRAVEL_DESK', 'Admin', 'TD001'),
('finance.admin@company.com', '$2b$10$hashedpassword', 'Finance', 'Admin', 'FINANCE', 'Finance', 'FIN001');
```

5. API Specifications

5.1 API Base Structure

Base URL: `https://api.rego.com/v1`
Authentication: Bearer Token (JWT)
Content-Type: `application/json`

5.2 Key API Endpoints

Authentication APIs

POST	/auth/register	Register new user (Admin only)
POST	/auth/login	User login
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Logout user
GET	/auth/profile	Get current user profile
PUT	/auth/profile	Update profile
POST	/auth/change-password	Change password

Travel Request APIs

POST	/travel-requests	Create travel request
GET	/travel-requests	Get all requests (with filters)
GET	/travel-requests/:id	Get request details
PUT	/travel-requests/:id	Update request
DELETE	/travel-requests/:id	Cancel request
POST	/travel-requests/:id/submit	Submit for approval
GET	/travel-requests/:id/timeline	Get approval timeline

Approval APIs

GET	/approvals/pending	Get pending approvals
GET	/approvals/history	Get approval history
POST	/approvals/:id/approve	Approve request
POST	/approvals/:id/reject	Reject request
POST	/approvals/:id/delegate	Delegate to another approver
PUT	/approvals/:id/add-comment	Add comment

Document APIs

POST	/documents/upload	Upload document
GET	/documents	Get all documents
GET	/documents/:id	Get document details
GET	/documents/:id/download	Download document
POST	/documents/:id/verify	Manual verification
DELETE	/documents/:id	Delete document
GET	/documents/:id/ocr-status	Check OCR processing status

Booking APIs

POST	/bookings	Create booking
GET	/bookings	Get all bookings
GET	/bookings/:id	Get booking details
PUT	/bookings/:id	Update booking
DELETE	/bookings/:id	Cancel booking

Expense APIs

POST	/expenses	Submit expense
GET	/expenses	Get all expenses
GET	/expenses/:id	Get expense details
PUT	/expenses/:id	Update expense
POST	/expenses/:id/approve	Approve expense
POST	/expenses/:id/reject	Reject expense
POST	/expenses/:id/reimburse	Process reimbursement

AI Recommendation APIs

POST	/ai/flight-recommendations	Get flight recommendations
POST	/ai/hotel-recommendations	Get hotel recommendations
POST	/ai/document-extract	Extract data via OCR
POST	/ai/expense-anomaly-check	Check expense anomalies

Notification APIs

GET	/notifications	Get user notifications
PUT	/notifications/:id/read	Mark as read
GET	/notifications/unread-count	Get unread count
POST	/notifications/preferences	Update notification preferences

5.3 Sample API Request/Response

POST /travel-requests

Request:

```
json
{
  "travelType": "INTERNATIONAL",
  "purpose": "Client meeting in New York",
  "destinationCountry": "USA",
  "destinationCity": "New York",
  "departureDate": "2025-12-15",
  "returnDate": "2025-12-20",
  "estimatedCost": 150000,
  "currency": "INR",
  "specialRequirements": "Vegetarian meals",
  "emergencyContactName": "Jane Doe",
  "emergencyContactPhone": "+919876543210",
  "isUrgent": false
}
```

Response (201):

```
json
{
  "success": true,
  "data": {
    "requestId": "550e8400-e29b-41d4-a716-446655440000",
    "requestNumber": "TR-2025-00123",
    "status": "DRAFT",
    "createdAt": "2025-11-12T22:08:42Z"
  },
  "message": "Travel request created successfully"
}
```

5.4 Error Response Format

```
json
```

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": [
      {
        "field": "departureDate",
        "message": "Departure date must be in the future"
      }
    ]
  }
}
```

HTTP Status Codes:

- (200) - Success
- (201) - Created
- (400) - Bad Request
- (401) - Unauthorized
- (403) - Forbidden
- (404) - Not Found
- (409) - Conflict
- (422) - Validation Error
- (500) - Internal Server Error

6. Frontend Architecture

6.1 Project Structure

```
rego-frontend/
├── public/
│   ├── index.html
│   └── favicon.ico
├── src/
│   ├── components/           # Reusable components
│   │   ├── common/
│   │   │   ├── Button/
│   │   │   ├── Input/
│   │   │   ├── Modal/
│   │   │   └── Loader/
```

```
| | | | — Toast/
| | | | — Card/
| | | | — layout/
| | | | — Header/
| | | | — Sidebar/
| | | | — Footer/
| | | — features/
| | | | — TravelRequestCard/
| | | | — ApprovalCard/
| | | | — DocumentUploader/
| | | | — ExpenseForm/
| | | | — StatusBadge/
| | — pages/ # Page components
| | | — Dashboard/
| | | — TravelRequests/
| | | | — CreateRequest.jsx
| | | | — ViewRequest.jsx
| | | | — EditRequest.jsx
| | | | — RequestList.jsx
| | | — Approvals/
| | | | — PendingApprovals.jsx
| | | | — ApprovalHistory.jsx
| | | — Bookings/
| | | — Expenses/
| | | | — SubmitExpense.jsx
| | | | — ExpenseList.jsx
| | | — Documents/
| | | — Profile/
| | — features/ # Redux slices
| | | — auth/
| | | | — authSlice.js
| | | | — authAPI.js
| | | — travelRequests/
| | | | — travelRequestSlice.js
| | | | — travelRequestAPI.js
| | | — approvals/
| | | — expenses/
| | — hooks/ # Custom hooks
| | | — useAuth.js
| | | — useNotification.js
| | | — useDocumentUpload.js
| | | — useDebounce.js
| | — services/ # API services
| | | — api.js # Axios instance
| | | — authService.js
| | | — travelService.js
| | | — approvalService.js
```



```
| | |— documentService.js
| | |— expenseService.js
| |— store/          # Redux store
| |   |— store.js
| |— routes/         # Route configuration
| |   |— AppRoutes.jsx
| |   |— PrivateRoute.jsx
| |— utils/          # Utility functions
| |   |— dateFormatter.js
| |   |— currencyFormatter.js
| |   |— validators.js
| |   |— constants.js
| |— styles/         # Global styles
| |   |— theme.js
| |   |— globalStyles.css
| |— App.jsx
| |— index.jsx
|— .env
|— package.json
|— README.md
```

6.2 Key Component Examples

Dashboard Component

jsx

```
// pages/Dashboard/Dashboard.jsx
```

```
import React, { useEffect } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { Grid, Paper, Typography, Box } from '@mui/material';
import StatCard from './components/StatCard';
import RecentRequests from './components/RecentRequests';
import PendingApprovals from './components/PendingApprovals';
```

```
const Dashboard = () => {
  const dispatch = useDispatch();
  const { user } = useSelector(state => state.auth);
  const { stats, loading } = useSelector(state => state.dashboard);
```

```
  useEffect(() => {
    dispatch(fetchDashboardStats());
  }, [dispatch]);
```

```
  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4" gutterBottom>
        Welcome back, {user.firstName}!
      </Typography>
```

```
      <Grid container spacing={3}>
        <Grid item xs={12} sm={6} md={3}>
          <StatCard
            title="Active Requests"
            value={stats.activeRequests}
            icon="flight"
            color="primary"
          />
```

```
        </Grid>
```

```
        <Grid item xs={12} sm={6} md={3}>
          <StatCard
            title="Pending Approvals"
            value={stats.pendingApprovals}
            icon="pending"
            color="warning"
          />
```

```
        </Grid>
```

```
        <Grid item xs={12} sm={6} md={3}>
          <StatCard
            title="Upcoming Trips"
            value={stats.upcomingTrips}
            icon="schedule"
            color="info"
          />
```

```

    />
  </Grid>
  <Grid item xs={12} sm={6} md={3}>
    <StatCard
      title="Pending Reimbursements"
      value={` ₹$ {stats.pendingAmount.toLocaleString()} `}
      icon="money"
      color="success"
    />
  </Grid>

  <Grid item xs={12} md={8}>
    <Paper sx={{ p: 2 }}>
      <RecentRequests />
    </Paper>
  </Grid>

  {[ 'MANAGER', 'AVP', 'SVP', 'CHRO' ].includes(user.role) && (
    <Grid item xs={12} md={4}>
      <Paper sx={{ p: 2 }}>
        <PendingApprovals />
      </Paper>
    </Grid>
  )}
</Grid>
</Box>
);
};

export default Dashboard;

```

Travel Request Form

```

jsx

```

```
// pages/TravelRequests/CreateRequest.jsx
```

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Formik, Form, Field } from 'formik';
import * as Yup from 'yup';
import {
  TextField, Select, MenuItem, Button, Grid,
  Paper, Stepper, Step, StepLabel, Box
} from '@mui/material';
import DocumentUploader from '../components/features/DocumentUploader';
import { createTravelRequest } from '../services/travelService';
import { useNotification } from '../hooks/useNotification';
```

```
const validationSchema = Yup.object({
  travelType: Yup.string().required('Required'),
  purpose: Yup.string().required('Required').min(10, 'Minimum 10 characters'),
  destinationCountry: Yup.string().required('Required'),
  destinationCity: Yup.string().required('Required'),
  departureDate: Yup.date()
    .required('Required')
    .min(new Date(), 'Must be future date'),
  returnDate: Yup.date()
    .required('Required')
    .min(Yup.ref('departureDate'), 'Must be after departure'),
  estimatedCost: Yup.number()
    .required('Required')
    .positive('Must be positive')
});
```

```
const CreateRequest = () => {
  const navigate = useNavigate();
  const { showSuccess, showError } = useNotification();
  const [activeStep, setActiveStep] = useState(0);
  const [requestId, setRequestId] = useState(null);

  const steps = ['Travel Details', 'Upload Documents', 'Review & Submit'];

  const handleSubmit = async (values) => {
    try {
      const response = await createTravelRequest(values);
      setRequestId(response.data.requestId);
      showSuccess('Travel request created successfully');
      setActiveStep(1);
    } catch (error) {
      showError('Failed to create request');
    }
  }
}
```

```

};

return (
  <Paper sx={{ p: 3 }}>
    <Stepper activeStep={activeStep} sx={{ mb: 4 }}>
      {steps.map(label => (
        <Step key={label}>
          <StepLabel>{label}</StepLabel>
        </Step>
      ))}
    </Stepper>

    <Formik
      initialValues={{
        travelType: "",
        purpose: "",
        destinationCountry: "",
        destinationCity: "",
        departureDate: "",
        returnDate: "",
        estimatedCost: "",
        currency: 'INR'
      }}
      validationSchema={validationSchema}
      onSubmit={handleSubmit}
    >
      ({ values, errors, touched, handleChange }) => (
        <Form>
          {activeStep === 0 && (
            <Grid container spacing={2}>
              <Grid item xs={12} md={6}>
                <Select
                  name="travelType"
                  value={values.travelType}
                  onChange={handleChange}
                  fullWidth
                  error={touched.travelType && errors.travelType}
                >
                  <MenuItem value="DOMESTIC">Domestic</MenuItem>
                  <MenuItem value="INTERNATIONAL">International</MenuItem>
                </Select>
              </Grid>

              <Grid item xs={12}>
                <TextField
                  name="purpose"
                  label="Purpose of Travel"

```

```
        multiline
        rows={3}
        fullWidth
        value={values.purpose}
        onChange={handleChange}
        error={touched.purpose && Boolean(errors.purpose)}
        helperText={touched.purpose && errors.purpose}
      />
    </Grid>
```

```
    <Grid item xs={12} md={6}>
      <TextField
        name="destinationCountry"
        label="Destination Country"
        fullWidth
        value={values.destinationCountry}
        onChange={handleChange}
        error={touched.destinationCountry && errors.destinationCountry}
      />
    </Grid>
```

```
    <Grid item xs={12} md={6}>
      <TextField
        name="destinationCity"
        label="Destination City"
        fullWidth
        value={values.destinationCity}
        onChange={handleChange}
      />
    </Grid>
```

```
    <Grid item xs={12} md={6}>
      <TextField
        name="departureDate"
        label="Departure Date"
        type="date"
        fullWidth
        InputLabelProps={{ shrink: true }}
        value={values.departureDate}
        onChange={handleChange}
        error={touched.departureDate && errors.departureDate}
      />
    </Grid>
```

```
    <Grid item xs={12} md={6}>
      <TextField
        name="returnDate"
```

```

        label="Return Date"
        type="date"
        fullWidth
        InputLabelProps={{ shrink: true }}
        value={values.returnDate}
        onChange={handleChange}
    />
</Grid>

<Grid item xs={12} md={6}>
    <TextField
        name="estimatedCost"
        label="Estimated Cost"
        type="number"
        fullWidth
        value={values.estimatedCost}
        onChange={handleChange}
    />
</Grid>
</Grid>
)}

{activeStep === 1 && (
    <DocumentUploader requestId={requestId} />
)}

{activeStep === 2 && (
    <Box>
        /* Review section - display all entered data */
        <Typography variant="h6">Review Your Request</Typography>
        /* Display summary */
    </Box>
)}

<Box sx={{ display: 'flex', justifyContent: 'space-between', mt: 3 }}>
    <Button
        onClick={() => setActiveStep(prev => prev - 1)}
        disabled={activeStep === 0}
    >
        Back
    </Button>
    <Button
        type={activeStep === steps.length - 1 ? 'submit' : 'button'}
        variant="contained"
        onClick={() => {
            if (activeStep < steps.length - 1) {
                setActiveStep(prev => prev + 1);
            }
        }}
    />
</Box>

```

```
    }  
  }  
  >  
    {activeStep === steps.length - 1 ? 'Submit' : 'Next'}  
  </Button>  
</Box>  
</Form>  
  )}  
</Formik>  
</Paper>  
);  
};  
  
export default CreateRequest;
```

6.3 State Management (Redux)

javascript


```
// features/travelRequests/travelRequestSlice.js
```

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
```

```
import { travelService } from '../services/travelService';
```

```
export const fetchTravelRequests = createAsyncThunk(
```

```
  'travelRequests/fetchAll',
```

```
  async (filters) => {
```

```
    const response = await travelService.getAll(filters);
```

```
    return response.data;
```

```
  }
```

```
);
```

```
export const createTravelRequest = createAsyncThunk(
```

```
  'travelRequests/create',
```

```
  async (requestData) => {
```

```
    const response = await travelService.create(requestData);
```

```
    return response.data;
```

```
  }
```

```
);
```

```
const travelRequestSlice = createSlice({
```

```
  name: 'travelRequests',
```

```
  initialState: {
```

```
    requests: [],
```

```
    currentRequest: null,
```

```
    loading: false,
```

```
    error: null
```

```
  },
```

```
  reducers: {
```

```
    setCurrentRequest: (state, action) => {
```

```
      state.currentRequest = action.payload;
```

```
    },
```

```
    clearError: (state) => {
```

```
      state.error = null;
```

```
    }
```

```
  },
```

```
  extraReducers: (builder) => {
```

```
    builder
```

```
      .addCase(fetchTravelRequests.pending, (state) => {
```

```
        state.loading = true;
```

```
        state.error = null;
```

```
      })
```

```
      .addCase(fetchTravelRequests.fulfilled, (state, action) => {
```

```
        state.loading = false;
```

```
        state.requests = action.payload.requests;
```

```
      })
```

```
.addCase(fetchTravelRequests.rejected, (state, action) => {  
  state.loading = false;  
  state.error = action.error.message;  
});  
}  
});  
  
export const { setCurrentRequest, clearError } = travelRequestSlice.actions;  
export default travelRequestSlice.reducer;
```

7. Authentication & Security

7.1 JWT Authentication Flow

1. User enters credentials → Frontend sends to /auth/login
2. Backend validates credentials
3. If valid: Generate JWT access token (15 min) + Refresh token (7 days)
4. Frontend stores tokens in localStorage/httpOnly cookie
5. All subsequent API calls include: Authorization: Bearer <token>
6. Backend middleware verifies JWT on each request
7. If access token expired: Use refresh token to get new one
8. On logout: Invalidate refresh token

7.2 Backend Auth Middleware

javascript

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];

    if (!token) {
      return res.status(401).json({
        success: false,
        error: { message: 'No token provided' }
      });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { userId, email, role }
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      error: { message: 'Invalid or expired token' }
    });
  }
};

module.exports = authMiddleware;
```

7.3 Role-Based Access Control (RBAC)

javascript

```
// middleware/rbac.js
const checkRole = (...allowedRoles) => {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        error: { message: 'Insufficient permissions' }
      });
    }
    next();
  };
};
```

```
// Usage
router.post('/travel-requests',
  authMiddleware,
  checkRole('EMPLOYEE'),
  createRequest
);

router.put('/approvals/:id/approve',
  authMiddleware,
  checkRole('MANAGER', 'AVP', 'SVP', 'CHRO'),
  approveRequest
);
```

7.4 Frontend Axios Interceptor

```
javascript
```

```
// services/api.js
import axios from 'axios';
import authService from './authService';

const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
  headers: { 'Content-Type': 'application/json' }
});

// Request interceptor - add token
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('accessToken');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor - handle token refresh
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;

      try {
        await authService.refreshToken();
        const newToken = localStorage.getItem('accessToken');
        originalRequest.headers.Authorization = `Bearer ${newToken}`;
        return api(originalRequest);
      } catch (refreshError) {
        window.location.href = '/login';
        return Promise.reject(refreshError);
      }
    }

    return Promise.reject(error);
  }
);
```

```
export default api;
```

8. AI & OCR Integration

8.1 Azure Document Intelligence for OCR

```
javascript
```

```
// services/ocrService.js
```

```
const { DocumentAnalysisClient, AzureKeyCredential } =
  require('@azure/ai-form-recognizer');

const endpoint = process.env.AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT;
const apiKey = process.env.AZURE_DOCUMENT_INTELLIGENCE_KEY;

const client = new DocumentAnalysisClient(
  endpoint,
  new AzureKeyCredential(apiKey)
);

const extractPassportData = async (documentUrl) => {
  try {
    const poller = await client.beginAnalyzeDocument(
      'prebuilt-idDocument',
      documentUrl
    );
    const result = await poller.pollUntilDone();

    const doc = result.documents[0];

    return {
      success: true,
      data: {
        passportNumber: doc?.fields?.DocumentNumber?.value,
        fullName: `${doc?.fields?.FirstName?.value} ${doc?.fields?.LastName?.value}`,
        dateOfBirth: doc?.fields?.DateOfBirth?.value,
        expiryDate: doc?.fields?.DateOfExpiration?.value,
        nationality: doc?.fields?.Nationality?.value
      },
      confidence: doc?.fields?.DocumentNumber?.confidence || 0
    };
  } catch (error) {
    return { success: false, error: error.message };
  }
};

const extractReceiptData = async (documentUrl) => {
  try {
    const poller = await client.beginAnalyzeDocument(
      'prebuilt-receipt',
      documentUrl
    );
    const result = await poller.pollUntilDone();
  }
};
```

```
const receipt = result.documents[0];

return {
  success: true,
  data: {
    merchantName: receipt?.fields?.MerchantName?.value,
    transactionDate: receipt?.fields?.TransactionDate?.value,
    total: receipt?.fields?.Total?.value,
    currency: receipt?.fields?.Total?.valueCurrency,
    items: receipt?.fields?.Items?.values?.map(item => ({
      description: item.properties?.Description?.value,
      price: item.properties?.Price?.value
    })))
  },
  confidence: receipt?.fields?.Total?.confidence || 0
};

} catch (error) {
  return { success: false, error: error.message };
}

};

module.exports = { extractPassportData, extractReceiptData };
```

8.2 Document Upload & OCR Flow

javascript


```
// controllers/documentController.js
```

```
const uploadDocument = async (req, res) => {  
  try {  
    const { requestId, documentType } = req.body;  
    const file = req.file;  
  
    // 1. Upload to Azure Blob  
    const blobUrl = await uploadToBlob(file);  
  
    // 2. Create document record  
    const document = await Document.create({  
      requestId,  
      uploadedBy: req.user.userId,  
      documentType,  
      fileName: file.originalname,  
      blobStorageUrl: blobUrl,  
      ocrStatus: 'PROCESSING'  
    });  
  
    // 3. Trigger OCR (async)  
    processOCR(document.documentId, documentType, blobUrl);  
  
    res.status(201).json({  
      success: true,  
      data: { documentId: document.documentId, ocrStatus: 'PROCESSING' }  
    });  
  } catch (error) {  
    res.status(500).json({ success: false, error: { message: error.message } });  
  }  
};
```

```
const processOCR = async (documentId, documentType, blobUrl) => {  
  try {  
    let ocrResult;  
  
    if (documentType === 'PASSPORT' || documentType === 'VISA') {  
      ocrResult = await extractPassportData(blobUrl);  
    } else if (documentType === 'RECEIPT') {  
      ocrResult = await extractReceiptData(blobUrl);  
    }  
  
    if (ocrResult.success && ocrResult.confidence > 0.7) {  
      await Document.update(documentId, {  
        ocrStatus: 'SUCCESS',  
        ocrExtractedData: ocrResult.data,  
        ocrConfidenceScore: ocrResult.confidence,  
      });  
    }  
  }  
};
```

```
        verificationStatus: 'VERIFIED'
    });
    sendNotification(documentId, 'DOCUMENT_VERIFIED');
  } else {
    await Document.update(documentId, {
      ocrStatus: 'FAILED',
      verificationStatus: 'MANUAL_REVIEW'
    });
    sendNotification(documentId, 'DOCUMENT_VERIFICATION_FAILED');
  }
} catch (error) {
  console.error('OCR error:', error);
}
};
```

8.3 AI Flight Recommendations

javascript

```
// services/flightRecommendationService.js
```

```
const axios = require('axios');
```

```
const getFlightRecommendations = async ({ origin, destination, departureDate, returnDate }) => {  
  try {
```

```
    // Get Amadeus token
```

```
    const tokenRes = await axios.post(  
      'https://test.api.amadeus.com/v1/security/oauth2/token',  
      new URLSearchParams({  
        grant_type: 'client_credentials',  
        client_id: process.env.AMADEUS_API_KEY,  
        client_secret: process.env.AMADEUS_API_SECRET  
      })  
    );
```

```
    // Search flights
```

```
    const flightRes = await axios.get(  
      'https://test.api.amadeus.com/v2/shopping/flight-offers',  
      {  
        headers: { Authorization: `Bearer ${tokenRes.data.access_token}` },  
        params: {  
          originLocationCode: origin,  
          destinationLocationCode: destination,  
          departureDate,  
          returnDate,  
          adults: 1,  
          max: 10  
        }  
      }  
    );
```

```
    // Categorize by cost
```

```
    const sorted = flightRes.data.data.sort((a, b) =>  
      parseFloat(a.price.total) - parseFloat(b.price.total)  
    );
```

```
    const third = Math.ceil(sorted.length / 3);
```

```
    return {  
      success: true,  
      recommendations: {  
        lowCost: {  
          flights: sorted.slice(0, third).slice(0, 3).map(formatFlight),  
          insight: 'Best value options with potential layovers'  
        },  
        midCost: {
```

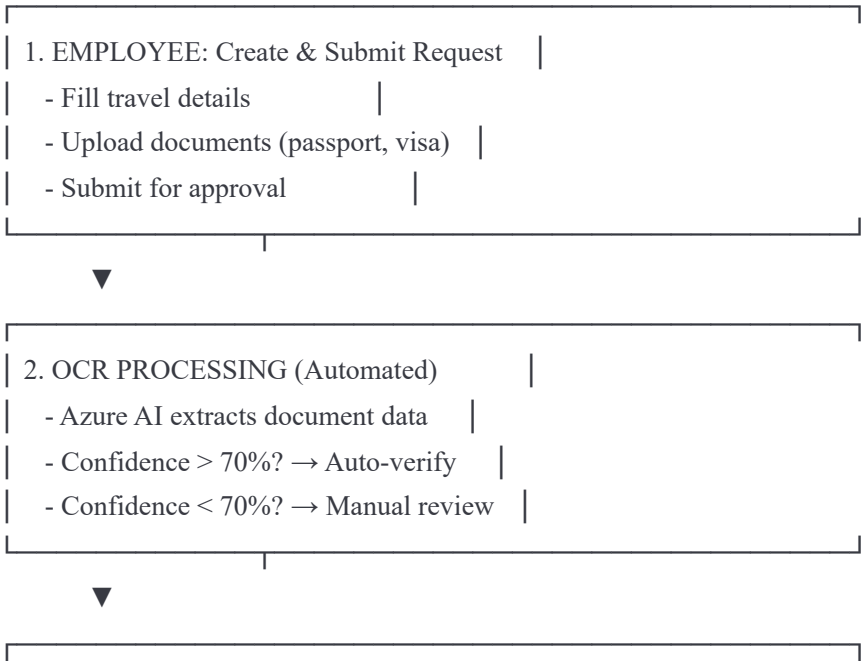
```
    flights: sorted.slice(third, third * 2).slice(0, 3).map(formatFlight),
    insight: 'Balanced timing and cost'
  },
  highCost: {
    flights: sorted.slice(third * 2).slice(0, 3).map(formatFlight),
    insight: 'Premium options with shortest duration'
  }
}
};

} catch (error) {
  return { success: false, error: error.message };
}
};

const formatFlight = (flight) => {
  const outbound = flight.itineraries[0];
  return {
    airline: outbound.segments[0].carrierCode,
    price: parseFloat(flight.price.total),
    currency: flight.price.currency,
    duration: outbound.duration.replace('PT', '').toLowerCase(),
    stops: outbound.segments.length - 1,
    departure: outbound.segments[0].departure.at,
    arrival: outbound.segments[outbound.segments.length - 1].arrival.at
  };
};
```

9. Complete Workflows

9.1 End-to-End Travel Request Workflow



3. MANAGER: First Approval (Level 1)

- Reviews request & documents
- Approve / Reject / Request changes
- Add budget constraints



4. AVP: Second Approval (Level 2)

- Budget verification
- Policy compliance check
- Approve / Reject



5. SVP: Third Approval (Level 3)

- Strategic alignment review
- High-value trip authorization
- Approve / Reject



6. CHRO: Final Approval (Level 4)

- Final authorization
- International travel sign-off
- Approve / Reject



7. TRAVEL DESK: Booking

- Get AI flight/hotel recommendations
- Book flights, hotels, insurance
- Process visa applications
- Upload booking confirmations
- Notify employee



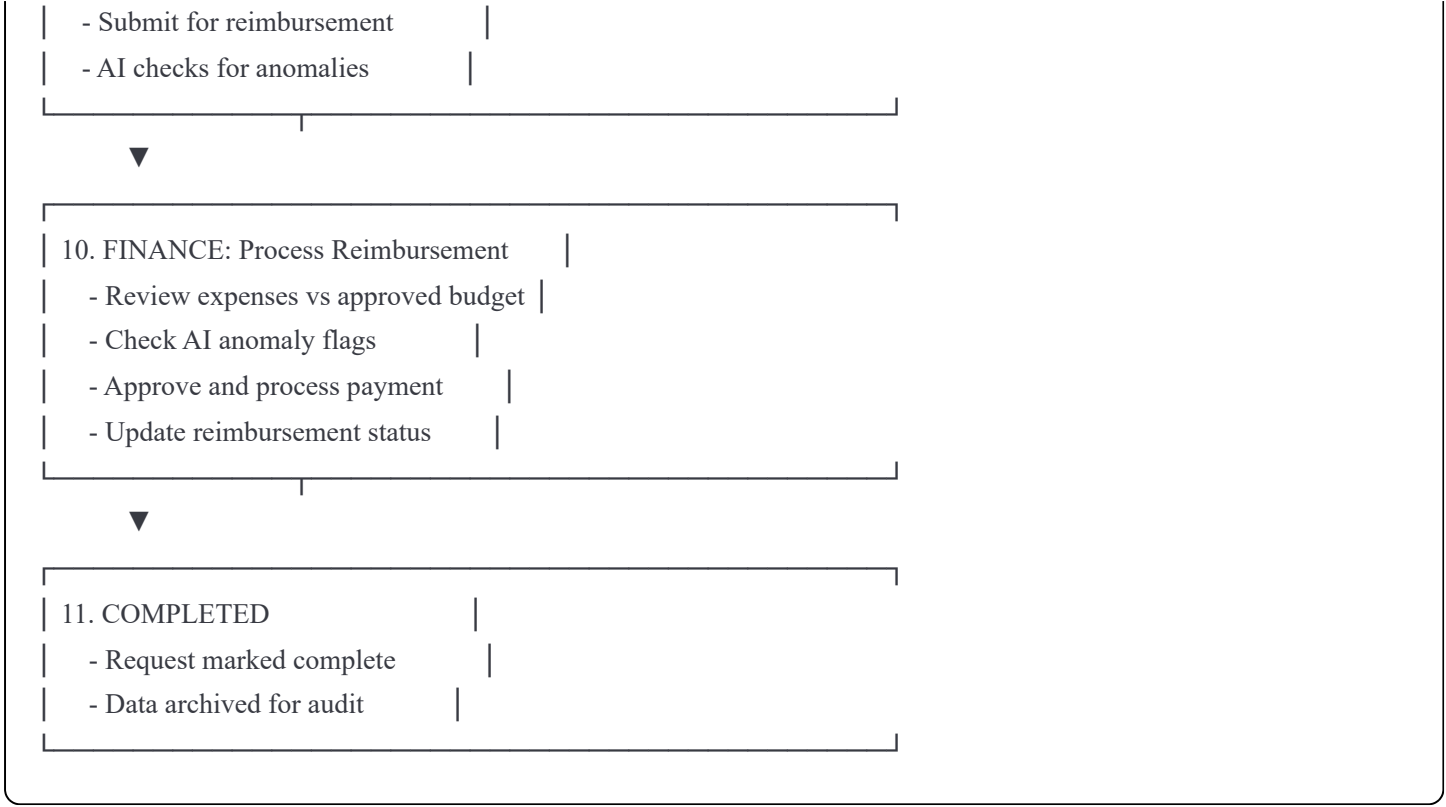
8. EMPLOYEE: Travel & Document

- Access all booking details in app
- Travel to destination
- Keep all receipts for expenses



9. EMPLOYEE: Submit Expenses

- Upload receipts (OCR extracts data)
- Enter expense details



9.2 Document Verification Flow





9.3 Approval Chain Decision Logic

```
javascript
// Approval chain logic
function determineApprovalChain(request) {
  const chain = ['MANAGER']; // Always starts with manager

  if (request.travelType === 'INTERNATIONAL') {
    chain.push('AVP', 'SVP', 'CHRO'); // All 4 levels
  } else if (request.travelType === 'DOMESTIC' && request.estimatedCost > 50000) {
    chain.push('AVP'); // Manager + AVP only
  }

  return chain;
}
```

10. Development Roadmap

10.1 Week-by-Week Plan

Week 1: Foundation (Days 1-7)

Goal: Project setup, authentication, basic UI

Day	Task	Owner	Deliverable
1-2	Project setup (Frontend + Backend)	Dev 1, Dev 2	Running dev environments
2-3	Database schema implementation	Dev 1	All tables created
3-5	Authentication system	Dev 1, Dev 2	Login/logout working
5-7	Basic UI layout & navigation	Dev 2, Dev 3	Dashboard skeleton

Week 1 Demo: Login system with empty dashboard

Week 2: Core Features (Days 8-14)

Goal: Travel request & approval workflow

Day	Task	Owner	Deliverable
8-10	Travel request CRUD	Dev 3	Create/view requests
10-12	Document upload	Dev 3, Dev 4	File upload works
12-14	Approval workflow	Dev 1, Dev 3	Multi-level approvals

Week 2 Demo: End-to-end request submission & approval

Week 3: AI Integration (Days 15-21)

Goal: OCR and AI recommendations

Day	Task	Owner	Deliverable
15-17	Azure Document Intelligence	Dev 4	OCR processing
17-18	Manual verification UI	Dev 3, Dev 4	Verification interface
19-21	AI travel recommendations	Dev 4	Flight/hotel suggestions

Week 3 Demo: OCR verification + AI recommendations

Week 4: Bookings & Expenses (Days 22-28)

Goal: Complete booking & expense management

Day	Task	Owner	Deliverable
22-24	Booking management	Dev 3	Travel desk portal
25-27	Expense tracking	Dev 4	Expense submission
27-28	Notifications	Dev 1	Email + SMS

Week 4 Demo: Complete expense tracking

Week 5: Polish & Deploy (Days 29-35)

Goal: Testing, fixes, deployment

Day	Task	Owner	Deliverable
29-31	Testing & bug fixes	Dev 5 + All	Test reports
31-32	UI/UX improvements	Dev 2	Polished UI
33	Documentation	All	Complete docs
34-35	Deployment	Dev 1, Dev 2	Live application

Week 5 Demo: Production-ready app

11. Team Assignment & Responsibilities

11.1 Role Distribution

Developer 1: Backend Lead

- Database schema & migrations
- Authentication & authorization
- Core API endpoints
- Azure service integrations
- Backend deployment

Key Deliverables:

- All API endpoints functional
- Database properly structured
- Authentication working
- Azure services integrated

Developer 2: Frontend Lead

- React project setup
- Component library
- Redux state management
- UI/UX implementation
- Frontend deployment

Key Deliverables:

- All pages created
- Reusable components
- State management setup
- Responsive design

Developer 3: Full Stack - Travel Module

- Travel request (full stack)
- Approval workflow
- Document upload
- Travel desk portal

- Booking interface

Key Deliverables:

- Complete travel flow
- Approval chain working
- Document management
- Booking system

Developer 4: Full Stack - AI & Expenses

- AI recommendation integration
- OCR processing workflow
- Expense management
- Finance dashboard
- Anomaly detection

Key Deliverables:

- AI recommendations working
- OCR functional
- Expense system complete
- Reimbursement flow

Developer 5: QA + Support Developer

- Test case creation
- Manual & automated testing
- Bug tracking & reporting
- Support other developers
- Code reviews

Key Deliverables:

- Test coverage >75%
- Bug reports
- Quality assurance
- Documentation help

11.2 Daily Standup (15 min)**Format:**

1. What did you accomplish yesterday?
2. What will you work on today?
3. Any blockers?

Time: 10:00 AM daily

11.3 Code Review Process

- All code via pull requests on GitHub
- Minimum 1 approval required
- Run tests before requesting review
- Dev 5 does final review before merging

12. Testing Strategy

12.1 Backend Testing (Jest + Supertest)

```
javascript
```

// Example: Travel Request API Test

```
describe('Travel Request API', () => {  
  let authToken;  
  
  beforeAll(async () => {  
    const response = await request(app)  
      .post('/api/auth/login')  
      .send({ email: 'test@company.com', password: 'Test123' });  
    authToken = response.body.data.token;  
  });  
  
  test('POST /travel-requests - should create request', async () => {  
    const response = await request(app)  
      .post('/api/travel-requests')  
      .set('Authorization', `Bearer ${authToken}`)  
      .send({  
        travelType: 'INTERNATIONAL',  
        purpose: 'Client meeting',  
        destinationCountry: 'USA',  
        destinationCity: 'New York',  
        departureDate: '2025-12-15',  
        returnDate: '2025-12-20',  
        estimatedCost: 150000  
      });  
  
    expect(response.status).toBe(201);  
    expect(response.body.data).toHaveProperty('requestId');  
  });  
});
```

12.2 Frontend Testing (React Testing Library)

javascript

// Example: Component Test

```
test('renders travel request card', () => {
  render(
    <Provider store={store}>
      <BrowserRouter>
        <TravelRequestCard request={mockRequest} />
      </BrowserRouter>
    </Provider>
  );

  expect(screen.getByText('TR-2025-00123')).toBeInTheDocument();
  expect(screen.getByText('New York, USA')).toBeInTheDocument();
});
```

12.3 E2E Testing (Cypress)

javascript

// Example: Complete Flow Test

```
describe('Travel Request Flow', () => {
  it('should create and submit request', () => {
    cy.visit('/login');
    cy.get('input[name="email"]').type('john@company.com');
    cy.get('input[name="password"]').type('Test123');
    cy.get('button[type="submit"]').click();

    cy.contains('New Travel Request').click();
    // Fill form...
    cy.contains('Submit').click();
    cy.contains('Request submitted successfully').should('be.visible');
  });
});
```

12.4 Test Coverage Goals

- **Backend:** 80% coverage
- **Frontend:** 70% coverage
- **Critical paths:** 100% coverage

13. Deployment Guide

13.1 Azure Services Required

1. **Azure App Service** (2 instances)
 - Frontend hosting

- Backend API hosting

2. Azure Database for PostgreSQL

- Flexible Server (Dev/Production tiers)

3. Azure Blob Storage

- Document & receipt storage

4. Azure Document Intelligence

- OCR processing service

5. Azure Communication Services

- Email & SMS notifications

13.2 Environment Variables

Backend .env:

```
bash

NODE_ENV=production
PORT=8080

DATABASE_URL=postgresql://user:pass@host:5432/rego_db

JWT_SECRET=your-secret-key
JWT_EXPIRE=15m
REFRESH_TOKEN_SECRET=your-refresh-secret
REFRESH_TOKEN_EXPIRE=7d

AZURE_STORAGE_ACCOUNT_NAME=regostorage
AZURE_STORAGE_ACCOUNT_KEY=your-key
AZURE_STORAGE_CONTAINER_NAME=documents

AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT=https://xxx.cognitiveservices.azure.com/
AZURE_DOCUMENT_INTELLIGENCE_KEY=your-key

AMADEUS_API_KEY=your-key
AMADEUS_API_SECRET=your-secret

AZURE_COMMUNICATION_CONNECTION_STRING=your-connection-string
```

Frontend .env:

```
bash

REACT_APP_API_URL=https://rego-backend.azurewebsites.net/api/v1
REACT_APP_ENV=production
```

13.3 Deployment Commands

```
bash

# Deploy Backend
az webapp up \
  --resource-group rego-resources \
  --name rego-backend \
  --runtime "NODE:20-lts"

# Deploy Frontend
npm run build
az webapp up \
  --resource-group rego-resources \
  --name rego-frontend \
  --html
```

13.4 CI/CD Pipeline (GitHub Actions)

```
yaml

# .github/workflows/deploy.yml
name: Deploy to Azure

on:
  push:
    branches: [ main ]

jobs:
  deploy-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Setup Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '20'
      - run: npm install
      - run: npm test
      - name: Deploy to Azure
        uses: azure/webapps-deploy@v2
        with:
          app-name: rego-backend
          publish-profile: ${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
```

14. Additional Features & Edge Cases

14.1 Additional Features to Consider

Phase 2 Enhancements:

1. **Multi-currency Support:** Auto-convert expenses to base currency
2. **Offline Mode:** Cache data for offline access
3. **Calendar Integration:** Sync trips with Outlook/Google Calendar
4. **Mobile App:** Native iOS/Android apps
5. **Advanced Analytics:** Dashboard with travel insights
6. **Travel Policy Engine:** Auto-reject non-compliant requests
7. **Preferred Vendors:** Track and suggest preferred airlines/hotels
8. **Group Travel:** Support for team travel requests
9. **Export Reports:** PDF/Excel export of expenses
10. **Approval Delegation:** Temporary delegation to other approvers

14.2 Edge Cases to Handle

1. **Duplicate Requests:** Same dates, destination → Warning
2. **Overlapping Travel:** Multiple requests for same period
3. **Budget Exceeded:** Flag when exceeding department budget
4. **Expired Documents:** Check passport expiry before travel date
5. **Cancelled Flights:** Handle mid-approval cancellations
6. **Emergency Travel:** Fast-track approval for urgent trips
7. **Partial Expenses:** Some expenses approved, some rejected
8. **Late Submissions:** Expenses submitted after deadline
9. **Multiple Approvers Absent:** Escalation to next level
10. **Document Reupload Limit:** Max 2 OCR attempts
11. **Large File Uploads:** Handle files >10MB
12. **Network Failures:** Retry logic for API calls
13. **Concurrent Edits:** Handle multiple users editing same request
14. **Time Zone Issues:** Store all dates in UTC
15. **Currency Conversion Rates:** Use real-time rates

14.3 Security Checklist

- ☐ JWT tokens with short expiry
 - ☐ HTTPS only (no HTTP)
 - ☐ SQL injection prevention (parameterized queries)
 - ☐ XSS protection (input sanitization)
 - ☐ CSRF protection
 - ☐ Rate limiting on APIs
 - ☐ File type validation on uploads
 - ☐ File size limits (max 10MB)
 - ☐ Secure password hashing (bcrypt)
 - ☐ Role-based access control
 - ☐ Audit logs for all actions
 - ☐ Encrypted database connections
 - ☐ Environment variables for secrets
 - ☐ Regular dependency updates
 - ☐ CORS configuration
-

Appendix

A. Useful Resources

Learning:

- React: <https://react.dev>
- Node.js: <https://nodejs.org/docs>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Azure Docs: <https://docs.microsoft.com/azure>

APIs:

- Amadeus: <https://developers.amadeus.com>
- Azure Document Intelligence: <https://learn.microsoft.com/azure/ai-services/document-intelligence>

Tools:

- Postman Collections: Create API collection for testing
- pgAdmin: Database management GUI
- VS Code Extensions: ESLint, Prettier, GitLens

B. Glossary

- **OCR:** Optical Character Recognition
- **JWT:** JSON Web Token

- **RBAC:** Role-Based Access Control
- **SSO:** Single Sign-On
- **API:** Application Programming Interface
- **CRUD:** Create, Read, Update, Delete
- **CI/CD:** Continuous Integration/Continuous Deployment
- **UUID:** Universally Unique Identifier
- **PNR:** Passenger Name Record

C. Contact & Support

For Questions:

- Technical Issues: Create GitHub issue
- Architecture Questions: Team Lead
- Azure Setup: Dev 1 (Backend Lead)
- UI/UX: Dev 2 (Frontend Lead)

☒ Final Checklist Before Hackathon Demo

- ☐ All 5 team members have dev environment running
- ☐ Database schema created and seeded
- ☐ Authentication working (login/logout)
- ☐ Can create travel request
- ☐ Can upload document
- ☐ OCR processes document
- ☐ Approval workflow functions
- ☐ Can submit expenses
- ☐ Reimbursement flow works
- ☐ Notifications send successfully
- ☐ UI is responsive
- ☐ No critical bugs
- ☐ Code is on GitHub
- ☐ Demo script prepared
- ☐ Presentation slides ready

Document Version: 1.0

Last Updated: November 12, 2025

Next Review: Weekly during development

Good luck with your hackathon! 🚀

